
Plantilla de Programación Competitiva

Universidad de Bogotá Jorge Tadeo Lozano

Facultad de Ciencias Naturales e Ingeniería

Semillero de Programación Competitiva

Maratón Nacional ACIS REDIS 2025

Elaborado por: Ludwig Alvarado y Nicolás Leitón

Bogotá D.C., 11 de octubre de 2025

Índice

1. Leer Input	1
1.1. Python	1
1.1.1. n inicial y luego casos	1
1.1.2. Terminar cuando es 0 0	1
1.1.3. Enumerar casos hasta EOF	1
1.1.4. Input matriz nxm	2
1.2. C++	2
1.2.1. n inicial y luego casos	2
1.2.2. Terminar cuando es 0 0	3
1.2.3. Leer líneas hasta EOF	3
1.2.4. Leer N líneas (string)	3
1.2.5. Leer vector	4
2. Output Format	4
2.1. Python	4
2.2. C++	5
3. Macros	5
3.1. Python	5
3.2. C++	6
4. Operaciones Bitwise	7
4.1. C++	7
5. Bitmask y Binarios	7
6. Swap dos números	8
7. Grafos	9
7.1. C++	9
7.1.1. Leer grafo no dirigido con peso	9
7.1.2. Leer grafo sin peso	9

1. Leer Input

1.1. Python

1.1.1. n inicial y luego casos

```
1 import sys
2 inputs = iter(sys.stdin.readlines())
3 TC = int(next(inputs))
4 for _ in range(TC):
5     print(sum(map(int, next(inputs).split())))
```

```
3
1 2
5 7
6 3
—— out
3
12
9
```

1.1.2. Terminar cuando es 0 0

```
1 import sys
2 for line in sys.stdin.readlines():
3     if line=='0 0\n': break
4     print(sum(map(int, line.split())))
```

```
1 2
5 7
6 3
0 0
1 1 <- no se procesa
—— out
3
12
9
```

1.1.3. Enumerar casos hasta EOF

```
1
2 import sys
3 for c, line in enumerate(sys.stdin.readlines(), 1):
4     print("Case %s: %s\n" % (c, sum(map(int, line.split()))))
```

```

1 2
5 7
6 3
—— out
Case 1: 3

Case 2: 12

Case 3: 9

```

1.1.4. Input matriz nxm

```

1
2 import sys
3 lines = sys.stdin.readlines()
4 i= 0
5 matrices = []
6
7 while i<len(lines):
8     # Leer dimensiones
9     n, m = map(int, lines[i].split())
10    i += 1
11
12    #Leer la matriz de tamaño n
13    matrix = [list(map(int, lines[i+j].split())) for j in
14               range(n)]
15    matrices.append(matrix)
16
17    i +=n

```

1.2. C++

1.2.1. n inicial y luego casos

```

1
2 int n; cin >> n;
3 while(n--){
4     int a, b; cin >> a >> b;
5     cout << (a + b) << endl;
6 }

```

```

3
1 2
5 7
6 3

```

—— out

3
12
9

1.2.2. Terminar cuando es 0 0

```
1 int a, b;  
2 while(cin >> a >> b, (a || b))  
3     cout << (a + b) << endl;
```

1 2
5 7
6 3
0 0
1 1 <— no se procesa
—— out
3
12
9

1.2.3. Leer líneas hasta EOF

```
1 string line;  
2 while(getline(cin, line)){  
3     stringstream ss(line);  
4     int x, sum = 0;  
5     while(ss >> x) sum += x;  
6     cout << sum << endl;  
7 }
```

3 4 5
10 20
7 8 9 10
—— out
12
30
34

1.2.4. Leer N líneas (string)

```
1 int N;  
2 if (!(cin >> N)) return 0;  
3 cin.ignore();
```

```

4 for(int i = 0; i < N; ++i){
5     string s;
6     getline(cin, s);
7 }

```

1.2.5. Leer vector

```

1 int n; cin >> n;
2
3 vector<int> v(n);
4 copy_n(istream_iterator<int>(cin), n, v.begin());

```

2. Output Format

2.1. Python

```

1 #Decimals output
2 text = "The price is {:.2f} dollars"
3 print(txt.format(45))
4 # The price is 45.00 dollar
5
6 #Use ',' as thousand separator:
7 txt = "the universe is {:,} years old "
8 print(txt.format(13800000000))
9 # The universe is 13,800,000,000 years old
10
11 # Convert to percentage format:
12 txt = "you scored {:.1%}"
13 print(txt.format(0.255))
14 # you scored 25.5%
15
16 txt = "you scored {:.0%}"
17 print(txt.format(0.255))
18 # you scored 26%
19
20
21 #convert number to binary
22
23 txt = "The binary version of {0} is {0:b}"
24 print(txt.format(5))
25 #The binary version of 5 is 101

```

2.2. C++

```
1
2 vector<int> v = {1, 2, 3, 4, 5};
3 copy(v.begin(), v.end(), ostream_iterator<int>(cout));
4 // 12345%
5
6 double x = 123.456789;
7 cout << fixed << setprecision(3);
8 cout << "Value: " << x << endl;
9 // Value: 123.457
10
11 cout << setw(10) << setfill('*') << 42 << endl;
12 // *****42
13
14 cout << boolalpha << (x > 100) << endl;
15 cout << hex << 255 << endl;
16 // true
17 // ff
18
19 vector<vector<int>> mat = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
20 for(auto &row : mat){
21     for (int i = 0; i < row.size(); ++i)
22         cout << (i ? " " : "") << row[i];
23     cout << endl;
24 }
25 /*
26 1 2 3
27 4 5 6
28 7 8 9
29 */
```

3. Macros

3.1. Python

```
1 def crear_matriz(n, m, valor =0):
2     return [[valor for _ in range(m)] for _ in range(n)]
3
4 def imprimir_matriz(mat):
5     for fila in mat:
6         print(*fila)
7
8 n, m = 3, 4
```

```

9  matriz= crear_matriz(n, m)
10 imprimir_matriz(matriz)
11
12 """
13 0 0 0 0
14 0 0 0 0
15 0 0 0 0
16 """

```

3.2. C++

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define fast ios::sync_with_stdio(false); cin.tie(nullptr);
   cout.tie(nullptr)
4  #define endl '\n'
5  #define pb push_back
6  #define all(x) (x).begin(), (x).end()
7  #define sz(x) (int)(x).size()
8  #define for0(i, n) for (int i = 0; i < (int)(n); ++i) // base
   0 index
9  #define for1(i, n) for (int i = 0; i < (int)(n); ++i) // base
   1 index
10 #define forc(i, l, r) for (int i = (int)(l); i <= (int)(r);
   ++i) // intervalo cerrado de l a r inclusivo
11 #define forr0(i, n) for (int i = (int)(n) - 1; i >= 0 ; --i)
   // al contrario hasta 0.
12 #define forr1(i, n) for (int i = (int)(n) - 1; i > 0; --i) //
   al contrario hasta 1
13
14 typedef long long ll;
15 typedef short int si;
16 typedef unsigned long long ull;
17 typedef long double ld;
18 typedef unsigned int ui;
19 typedef string str;
20 typedef pair<int, int> pii;
21 typedef pair<ll, ll> pll;
22 typedef vector<int> vi;
23
24
25 const int INFI = INT_MAX;
26 const long INFL = LONG_MAX;
27 const long long INFLl = LLONG_MAX;

```

4. Operaciones Bitwise

4.1. C++

```
1
2 // Macros para operaciones bitwise
3 #define isOn(S, j) (S & (1 << j))
4 #define setBit(S, j) (S |= (1 << j))
5 #define clearBit(S, j) (S &= ~(1 << j))
6 #define toggleBit(S, j) (S ^= (1 << j))
7 #define lowBit(S) (S & (-S))
8 #define setAll(S, n) (S = (1 << n)-1)
9
10 #define modulo(S, N) ((S) & (N-1)) // Retorna S % N, donde N
    es una potencia de 2
11 #define isPowerOfTwo(S) (!(S) & (S-1))
12 #define nearestPowerOfTwo(S) (1 << lround(log2(S)))
13 #define turnOffLastBit(S) ((S) & (S-1))
14 #define turnOnLastZero(S) ((S) | (S+1))
15 #define turnOffLastConsecutiveBits(S) ((S) & (S+1))
16 #define turnOnLastConsecutiveZeroes(S) ((S) | (S-1))
```

5. Bitmask y Binarios

```
1  ll binpow(ll a, ll b){
2      ll res = 1;
3      ll counter = 0;
4      while (b > 0){
5          bitset<16> binary(b);
6          if ( b& 1 ) res = res * a;
7          a + a * a;
8          counter++;
9          b >>=1;
10     }
11     return res;
12 }
13
14 ll binpowmod(ll a, ll b, ll m){
15     a \% = m;
16     ll res = 1;
17     while ( b > 0 ){
18         if ( b & 1 ) res = res * a \% m;
19         a = a * a \% m;
20         b >>= 1;
```

```

21     }
22     return res;
23 }
24
25
26 bitset<8> b1; // 00000000
27 bitset<8> b2(5); // 00000101
28 bitset<8> b3(string("1101")); // 00001101
29 bitset<8> b("10110010")
30 b[0] // 0 (least significant bit)
31 b[7] // 1 (most significant bit)
32 b[0] = 1; // Set bit 0 10110011
33 b.set(3); // set bit at index 3 10111011
34 b.reset(1); // reset bit at index 1
35 b.flip(2); // toggle bit at index 2 10111101
36 b.flip();
37
38 b.any() // True if any bit == 1
39 b.none() // true if all bits == 0
40 b.all() // true if all bits == 1
41 b.count() // number of bits == 1
42 b.size() // number of bits in bitset
43
44 b.to_ulong() // unsigned long
45 b.to_ullong() // unsigned long long
46 b.to_string() // string
47
48 // Obtener los subconjuntos de una mascara en binario
49 vector<int> subset_bitmask(int mask){
50     vector<int> subsets;
51     for (int subset = mask; subset; subset = (mask & (subset
52         -1))) subsets.push_back(subset);
53     return subsets;
54 }

```

6. Swap dos números

```

1 void swap_two_iters(vector<int> vec, int n){
2     const auto itr = find(all(vec), n);
3     auto indx = distance(vec.begin(), itr);
4     iter_swap(vec.begin() + n, itr);
5     iter_swap(vec.begin() + n, vec.begin() + indx);
6 }
7

```

```

8  int num = 2;
9  const auto itr = find(all(v), num);
10 auto indx = distance(v.begin(), itr);
11 iter_swap(v.begin() + num, v.begin() + indx);

```

7. Grafos

7.1. C++

7.1.1. Leer grafo no dirigido con peso

```

1  int n, m;
2  cin >> n >> m;
3
4  vector<vector<pair<int,int>>> adj(n + 1); // 1-indexed graph
5
6  for (int i = 0; i < m; ++i) {
7      int u, v, w;
8      cin >> u >> v >> w;
9      adj[u].push_back({v, w});
10     adj[v].push_back({u, w}); // Remover si el grafo es dirigido
11 }

```

7.1.2. Leer grafo sin peso

```

1  int n, m;
2  cin >> n >> m;
3
4  vector<vector<int>> adj(n + 1);
5
6  for (int i = 0; i < m; ++i) {
7      int u, v;
8      cin >> u >> v;
9      adj[u].push_back(v);
10     adj[v].push_back(u);
11 }

```